

Experiment Book 3

Inspect storage, manage processes, and package files

This draft contains setup and Experiment 1 only. Begin with folder usage and available filesystem space. Stop at the review checkpoint at the end of this draft.

Name: _____ Lab ID: _____ Section: _____

Get ready

Sign in with your own GitHub account. Open the Lab 3 link below and create a Codespace on **main**. Later, resume that same Codespace to continue. Books 1 and 2 use different repositories.

<https://codespaces.new/nalinabrol/terminal-lab-3?quickstart=1>

Wait for setup to finish. Choose **Trust Folder & Continue** if asked. A Bash terminal opens. If needed, use the Command Palette command **Lab 3: Focus Bash Terminal**. At the repository root, run:

```
bash terminal-lab-3/start.sh
```

Enter your instructor-assigned lab ID, not your email. Run the exact **cd** command printed by setup. Then run these commands one at a time:

```
pwd
cat identity.txt
ls
```

My lab root (the complete path printed by pwd):

Folder	Purpose
practice/	Original event exports. Keep these files unchanged.
challenge/	Original evidence for a short independent application.
work/	Your saved measurements, observations and working copies.

How to work: predict, run, inspect, explain. Start each section at your lab root. Use familiar terminal commands and nano; no solution scripts are needed. Save only inside work/.

Reusing an ID preserves every existing file; it does not reset an attempt. Save in nano with Ctrl+O, Enter, then exit with Ctrl+X. If the browser intercepts Ctrl+X, use F2 to exit nano. Use Control on a Mac too. Keep this Codespace for review; ignored lab-work/ is not a GitHub backup.

Which folder uses the most?

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

The campus event team has notices, posters and recordings. Before making another copy, inspect the space these folders already use. A folder name alone does not tell you its size.

```
cat practice/README.txt
ls practice
ls practice/recordings
```

The .bin files are simulated stored exports, not playable media. Measure their folders; do not open these binary files in cat or nano.

1A | Predict, then measure

I predict the largest folder is _____ because:

```
du -sh practice/notices practice/posters practice/recordings
```

du estimates the storage used by the files and folders you name. **-s** gives one summary per named path, including its contents. **-h** displays readable units. You can combine them as **-sh**. This command measures; it does not change the folders.

Folder	My displayed size, including its unit
notices	
posters	
recordings	

1B | Read the units before comparing

For these GNU commands, **K** means units of 1,024 bytes, **M** means 1,024 K, and **G** means 1,024 M. Displays are rounded. Compare the units as well as the numbers: a value in M may exceed a larger-looking number in K.

Which is larger: 900K or 2.0M? Explain:

1C | Save evidence

```
du -sh practice/notices practice/posters practice/recordings > work/guided-usage.txt
cat work/guided-usage.txt
nano work/observations.txt
```

Fill **guided_largest=** with only the largest folder name, after the equals sign. Leave the other lines for later. Save and exit. The report retains all three sizes and paths; your answer identifies the largest folder.

The > symbol replaces its destination file. It must point to work/, never to an original evidence file. Exact totals can include folder overhead; your classmate may see slightly different values.

How much space is available?

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

A **filesystem** is the storage area in which files and folders are organised. Your lab folder occupies only part of one filesystem. Available space belongs to that filesystem, not to a folder with its own separate allowance.

1D | Inspect the filesystem containing this folder

```
df -h .
```

df reports filesystem capacity and space usage. **-h** uses readable units. The dot **.** means your current folder; here it selects the filesystem containing your lab root. It does not ask **df** to add up the files in that folder.

Column	What to read
Filesystem	The filesystem/device label. Its name can vary.
Size	Total capacity reported for this filesystem.
Used	Space already used across the filesystem.
Avail	Space reported as available for use.
Use%	The reported percentage used, not the percentage free.
Mounted on	Where this filesystem is attached to the directory tree.

These values describe storage inside your remote Linux Codespace, not the free space on your own laptop. In Codespaces, the mount can be `/workspaces` or another path. Read what your output actually says.

1E | Save one snapshot and interpret it

```
df -h . > work/filesystem.txt
cat work/filesystem.txt
nano work/observations.txt
```

Fill **filesystem_size**, **filesystem_used**, **filesystem_available**, **filesystem_use_percent** and **filesystem_mounted_on** from your **saved** output. Keep the units, percent sign and full mount path. Save and exit.

Which column would you inspect before saving another large export?

Capacity, usage and available space can differ across students and change during the session. Do not copy someone else's numbers. Rounded values and filesystem accounting mean `Used + Avail` need not exactly equal `Size`. No exact free-space value is required.

Two commands, two questions

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

1F | Compare the same dot

```
du -sh .  
df -h .
```

du -sh . displays this size for my lab folder: _____

df -h . displays this filesystem Size: _____ and Avail: _____

Both commands use the dot. Why do they answer different questions?

1G | Predict a small, controlled change

The folder **work/copy-target/** starts empty. You will copy one supplied recording into it, keeping the original. Predict which folder measurement will increase. Will a rounded df display necessarily change?

My prediction and reason:

```
ls work/copy-target  
du -sh work/copy-target > work/copy-before.txt  
cp practice/recordings/recording.bin work/copy-target/  
du -sh work/copy-target > work/copy-after.txt  
cat work/copy-before.txt work/copy-after.txt  
du -sh practice/recordings  
du -sh .  
df -h .
```

Run the copy sequence once. If resuming after the copy, inspect your saved before/after reports instead of overwriting the before report. If the target already contains a file on your first attempt, ask your instructor for help.

What grew? What evidence shows that the original still exists?

A small copy can increase folder usage while rounded df values look unchanged. Other Codespace activity can also change filesystem usage. An unchanged Avail display does not prove that the copy used no storage. An empty directory may itself occupy a little space.

Inspect a new set of exports

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

The student magazine team needs the same kind of storage audit. Use fresh evidence in challenge/. Choose the commands yourself, drawing on the guided work.

```
cat challenge/brief.txt
ls challenge
```

Your task

1. Predict. Write down which of **design**, **notes** or **interviews** you expect to occupy the most storage.

My prediction and reason:

2. Measure. Use `du` to get one readable summary for each of the three folders. Save the three output lines as **work/challenge-usage.txt**. Do not measure only the combined challenge folder.

My command:

3. Inspect. Read your saved report. Compare the values with their units, then fill **challenge_largest=** in **work/observations.txt** with only the largest folder name.

The largest folder and the displayed size supporting my answer:

4. Apply. The team now asks whether this filesystem has space for another export. Run the appropriate command from your lab root. Write the command and the column you inspected.

Command and column:

My available-space reading, with its unit:

Why would measuring only the largest folder fail to answer that question?

Keep all originals in challenge/ unchanged. You do not need to copy or remove any challenge data. A folder can be the largest in this small dataset while using only a small fraction of filesystem capacity.

Make your evidence reviewable

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

Check saved work

```
cat work/observations.txt
python3 ../../terminal-lab-3/check.py
```

The checker reports **9 file checks**. PASS means a saved result meets that check. CHECK names something to inspect and gives a hint. It never repairs, uploads or submits your files. Correct the named result, save it, and run the checker again.

Evidence in work/	What it records
guided-usage.txt / challenge-usage.txt	Three folder summaries in each report.
filesystem.txt	The saved df header and filesystem row.
observations.txt	Your two folder choices and five filesystem fields.
copy-before.txt / copy-after.txt	The target folder measurement before/after the copy.
copy-target/recording.bin	The working copy; the original remains in practice/.

Understanding check - explain in your own words

1. A friend says "du -sh . tells me how much more I can save here." Correct the claim and give the command they need.

2. Does Used in df describe just your lab folder? Explain.

3. After your copy, Avail looks unchanged. What evidence shows the copy happened, and why might Avail look the same?

4. What do -s and -h each contribute to du -sh?

Review checkpoint: show your instructor the saved reports and your explanations. File checks do not prove understanding or independent work. This review draft ends with Experiment 1; there is no next experiment to complete yet.

Keep beside your terminal

Try a task before reading its hint. Examples below support Experiment 1 only.

Command	Question it answers
<code>du -sh FOLDER</code>	How much storage does this folder and its contents use?
<code>du -sh FOLDER_A FOLDER_B</code>	How much storage does each named folder use?
<code>du -sh .</code>	How much storage does my current folder and its contents use?
<code>df -h .</code>	What capacity, usage and availability does the filesystem containing my current folder report?

Familiar tools you will reuse

Command / keys	Reminder
<code>pwd / ls / cd PATH</code>	Show your location / list names / change folder.
<code>cat work/report.txt</code>	Read a saved text report.
<code>COMMAND > work/report.txt</code>	Save command output; replace that destination file.
<code>cp SOURCE DESTINATION</code>	Copy to the named destination; can overwrite a file.
<code>nano work/observations.txt</code>	Edit the observation lines. Keep their key names.
<code>Ctrl+O, Enter / Ctrl+X or F2</code>	In nano: save and confirm filename / exit.

Hints without the answers

Choosing the largest: compare units first. Do not sort the displayed size as ordinary text and assume that gives the storage order.

Reading df: follow each column heading vertically. Use the saved snapshot for the five observation fields, even if a fresh run has changed.

Independent report: use the three named challenge folders as separate `du` arguments. Save the report in `work/` and inspect it with `cat`.

Unexpected CHECK: confirm your location with `pwd`. Check spelling, units and destination paths. A rerun of setup preserves mistakes too. Ask for help if originals or before-copy evidence were changed.

Preserve progress: save editor changes and resume this same Codespace. Download `work/` for a separate copy if your instructor requests it. Deleting a Codespace can lose its unbacked-up work.

Command behaviour reference: [GNU Coreutils - File space usage](#). Linux outputs may vary; record observations from your own Codespace.